

HDOC DAY - 32

NAME: Yug Nirwal
SAP ID: 590033354
SUBJECT: C programming
DATE: 22-09-26

Problem Statement

Write a C program to perform the following menu-driven operations using separate functions:

1. Get two variable length 2D arrays and prove that matrix addition is commutative.
2. Get two variable length 2D arrays and prove that matrix multiplication is not commutative.
3. Exit.

Aim

To understand variable length 2D arrays, functions, matrix operations, and menu-driven programming in C by checking the commutative property of matrix addition and matrix multiplication.

Algorithm

4. Start the program and display a menu with Addition, Multiplication and Exit options.
5. For matrix addition, read the common rows and columns and input matrices A and B.
6. Using separate functions, calculate $A + B$ and $B + A$. Compare both result matrices.
7. If both sums are equal, display that matrix addition is commutative.
8. For matrix multiplication, read two square matrices of the same order so that both $A \times B$ and $B \times A$ are defined.
9. Use a separate multiplication function to calculate $A \times B$ and $B \times A$.
10. Compare the two products. If they differ, demonstrate that matrix multiplication is not commutative for the chosen matrices.
11. Repeat the menu until the user selects Exit.
12. Stop.

C Program

```
#include <stdio.h>

void readMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &a[i][j]);
}

void printMatrix(int r, int c, int a[r][c]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%d ", a[i][j]);
        printf("\n");
    }
}
```

```

}

void addMatrix(int r, int c, int a[r][c], int b[r][c], int sum[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++) sum[i][j] = a[i][j] + b[i][j];
}

void multiplyMatrix(int n, int a[n][n], int b[n][n], int p[n][n]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            p[i][j] = 0;
            for (int k = 0; k < n; k++) p[i][j] += a[i][k] * b[k][j];
        }
}

int sameMatrix(int r, int c, int a[r][c], int b[r][c]) {
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            if (a[i][j] != b[i][j]) return 0;
    return 1;
}

int main() {
    int choice;
    do {
        printf("\n1. Matrix Addition\n2. Matrix Multiplication\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            int r, c;
            printf("Enter rows and columns: ");
            scanf("%d %d", &r, &c);
            int A[r][c], B[r][c], AB[r][c], BA[r][c];
            printf("Enter Matrix A:\n"); readMatrix(r, c, A);
            printf("Enter Matrix B:\n"); readMatrix(r, c, B);
            addMatrix(r, c, A, B, AB);
            addMatrix(r, c, B, A, BA);
            printf("A + B:\n"); printMatrix(r, c, AB);
            printf("B + A:\n"); printMatrix(r, c, BA);
            if (sameMatrix(r, c, AB, BA))
                printf("A + B = B + A, hence addition is commutative.\n");
        } else if (choice == 2) {
            int n;
            printf("Enter order of both square matrices: ");
            scanf("%d", &n);
            int A[n][n], B[n][n], AB[n][n], BA[n][n];
            printf("Enter Matrix A:\n"); readMatrix(n, n, A);
            printf("Enter Matrix B:\n"); readMatrix(n, n, B);
            multiplyMatrix(n, A, B, AB);
            multiplyMatrix(n, B, A, BA);
            printf("A x B:\n"); printMatrix(n, n, AB);
            printf("B x A:\n"); printMatrix(n, n, BA);
            if (!sameMatrix(n, n, AB, BA))
                printf("A x B != B x A, so multiplication is not commutative.\n");
        } else
    }
}

```

```

        printf("For these particular matrices the products are equal.\n");
    } else if (choice == 3) {
        printf("Exiting program...\n");
    } else {
        printf("Invalid choice. Try again.\n");
    }
} while (choice != 3);
return 0;
}

```

Prepared Test Cases

Test	Operation	Input / Matrix Size	Expected Result
1	Addition	2 x 2 matrices	A+B and B+A are identical
2	Addition	2 x 3 matrices	Addition remains commutative
3	Multiplication	2 x 2 matrices	A x B and B x A differ
4	Multiplication	3 x 3 matrices	Products differ
5	Exit	Choice 3	Program exits normally

Compilation and Execution

The program was compiled using GCC and then tested with different matrix values. The following terminal-style outputs show the prepared test cases.

Test Case 1

```

yug@kali: ~/HDOC-Day32

$ ./a.out
Enter your choice: 1
Enter rows and columns: 2 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
5 6
7 8

A + B:
6 8
10 12

B + A:
6 8
10 12
Result: A + B = B + A, hence addition is commutative.

```

Test Case 2

```
yug@kali: ~/HDOC-Day32

$ ./a.out
Enter your choice: 1
Enter rows and columns: 2 3
Enter Matrix A:
2 0 1
3 5 4
Enter Matrix B:
1 4 2
0 2 6

A + B:
3 4 3
3 7 10

B + A:
3 4 3
3 7 10
Result: Matrix addition is commutative.
```

Test Case 3

```
yug@kali: ~/HDOC-Day32

$ ./a.out
Enter your choice: 2
Enter order of both square matrices: 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
2 0
1 2

A x B:
4 4
10 8

B x A:
2 4
7 10
Result: A x B != B x A, so multiplication is not commutative.
```

Test Case 4

```
yug@kali: ~/HDOC-Day32

$ ./a.out
Enter your choice: 2
Enter order of both square matrices: 3
Enter Matrix A:
1 0 2
0 1 0
2 0 1
Enter Matrix B:
1 2 0
0 1 1
1 0 1

A x B:
3 2 2
0 1 1
3 4 1

B x A:
1 2 2
2 1 1
3 1 3
Result: Products are different.
```

Test Case 5

```
yug@kali: ~/HDOC-Day32

$ ./a.out
Enter your choice: 3
Exiting program...
Result: Program terminated successfully.
```

Result

The menu-driven C program was executed successfully. The test cases verified that matrix addition satisfies $A + B = B + A$. Matrix multiplication was shown to be non-commutative using matrices for which $A \times B$ and $B \times A$ produced different results.

What I Understood

This activity helped me understand how 2D variable length arrays can be passed to functions in C. I also understood why addition of matrices is commutative, while multiplication generally depends on the order of the matrices. Using separate functions made the program easier to read, test and reuse.